

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	RAGAVI S	Reg. No.:	192565027
Section / Batch:	SLOT A	Date:	05-08-2026

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A digital computer performs signed integer arithmetic using 8-bit 2's complement representation. Consider the numbers -75 and $+38$. Analyze how the processor performs both addition and subtraction by converting the operands to binary, executing the operations step-by-step, detecting overflow (if any), and interpreting the final results in both binary and decimal forms.
2. A high-performance processor employs an 8-bit carry look-ahead adder (CLA) to enhance arithmetic speed. Given two binary operands, analyse how the generate and propagate signals are produced, how carry bits are calculated simultaneously, and compare the execution time and hardware complexity of both adders. Explain why CLA is widely used in modern processors.
3. A digital processing unit executes signed multiplication using Booth's algorithm. Multiply $+13$ and -6 , and analyze the algorithm step-by-step by showing the contents of the accumulator, multiplier, multiplicand, and Booth bit after each iteration. Explain the purpose of arithmetic right shifts and discuss how Booth's algorithm reduces the number of addition and subtraction operations.
4. A binary arithmetic unit supports both restoring and non-restoring division methods for signed integer operations. Analyze the division of the binary numbers $27 \div 5$ using each algorithm by showing the intermediate remainder values, quotient formation, and register contents after every iteration. Compare the two methods based on execution steps, computational complexity, and hardware efficiency, and justify which algorithm is more suitable for high-speed processors.
5. A floating-point processing unit represents real numbers using the IEEE 754 standard. Analyze the representation of the decimal number -42.375 in both single-precision (32-bit) and double-precision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between -42.375 and $+15.125$, including exponent alignment, mantissa operations, normalization, rounding, and the impact of precision on the final result.

Code of Conduct

I RAGAVI S / 192565027 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: RAGAVI S

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Question 1 — Signed Arithmetic: -75 and $+38$ (8-bit, 2's Complement)

1. 8-bit 2's complement Arithmetic.

a) $+38 + (-75)$ $\rightarrow$ Addition.

$$\Rightarrow 38 = 00100110$$

$$+75 = 01001011$$

2's complement of 75 = -75 .

$$\begin{array}{r} \Rightarrow \quad 01001011 \\ \underline{10110100} \\ 10110101 \rightarrow -75. \end{array}$$

$$\begin{array}{r} \Rightarrow \quad \overset{\textcircled{1}}{0}\overset{\textcircled{1}}{0}100110 \\ \underline{10110101} \\ 11011011 \\ \rightarrow (-ve) \end{array}$$

Taking complement of the result ;

$$\Rightarrow 00100100 = -37$$

$$\therefore 38 + (-75) = -37.$$

b) $-75 - (38)$ $\rightarrow$ Subtraction.

2's complement of 38 = -38

$$\begin{array}{r}
 00100110 \\
 \hline
 11011001 \\
 \hline
 11011010 = -38.
 \end{array}$$

$$\begin{array}{r}
 \Rightarrow 10110101 \\
 11011010 \\
 \hline
 11000111 \\
 \downarrow \rightarrow (-ve)
 \end{array}$$

Discard

$$\therefore \cancel{10001111} \Rightarrow 0$$

Taking 2's complement for result:

$$\begin{array}{r}
 \Rightarrow 10001111 \\
 \hline
 01110000 \\
 \hline
 01110001 \rightarrow -113
 \end{array}$$

$\therefore -75 - 38 = -113.$

$$\therefore 38 + (-75) = -37$$

$$-38 - 75 = -113.$$

Both results (-37 and -113) lie within the valid 8-bit signed range (-128 to +127), so they are correctly and validly represented.

Question 2 — Carry Look-Ahead Adder

2) $A = 01110110$; $B = 00101101$.

i	A_i	B_i	G_i	P_i
0	0	1	0	1
1	1	0	0	1
2	1	1	1	0
3	0	1	0	1
4	1	0	0	1
5	1	1	1	0
6	1	0	0	1
7	0	0	0	0

Sum bit $S_i = P_i \oplus C_i$

$\Rightarrow 10100011$, Carry out = 0

Result : $118 + 45 = 163$

$\Rightarrow 10100011$

Comparison: A ripple carry adder needs n gate-delays (one per bit) since each stage waits on the previous carry. A CLA computes all carries almost simultaneously (roughly 2–3 gate delays regardless of width) at the cost of extra gates for the expanded carry logic. This is why CLA is preferred in high-speed ALUs — the speed gain outweighs the extra hardware, especially as word length grows.

Question 3 — Booth's Algorithm

3) $M = 13 = 01101 \Rightarrow 13 \times (-6)$

$-M = 10011$

$Q = -6 = 11010$

$Q_{-1} = 0$

$A = 00000$

n	A	Q	Q ₋₁	Operation
5	00000	11010	0	Initial Value
	00000	11010	0	$Q_0Q_{-1} = 00 \rightarrow$ no operation
	00000	01101	0	ASR.
4	10011	01101	0	$Q_0Q_{-1} = 10 \rightarrow A = A - M$
	11001	10110	1	ASR.
3	00110	10110	1	$Q_0Q_{-1} = 01 \rightarrow A = A + M$
	00011	01011	0	ASR.
2	10110	01011	0	$Q_0Q_{-1} = 10 \rightarrow A = A - M$
	11011	00101	1	ASR.
1	11011	00101	1	$Q_0Q_{-1} = 11 \rightarrow$ no operation
	11101	10010	1	ASR

$\rightarrow$ Final AQ = 1110110010 $\rightarrow$ negative

$\therefore$ 2's Complement = 0001001110 = 78

$\therefore$ Result = -78.

$\therefore 13 \times (-6) = -78$

Purpose of arithmetic shift right (ASR): it shifts the AQ register right by one bit while preserving the sign bit (MSB), correctly repositioning the partial product for the next iteration without corrupting its sign.

Efficiency: Booth's algorithm inspects the pair (Q₀, Q₋₁) each cycle; whenever these bits are equal (00 or 11), no addition or subtraction is performed — only a shift. This means long runs of identical bits in the multiplier require no arithmetic operation at all, reducing the total number of adds/subtracts compared to a plain shift-and-add multiplier.

Question 4 — $27 \div 5$: Restoring and Non-Restoring Division

4) $27 = 11011$

$5 = 00101$

n	A	Q	Action.
5	0000 0001 11100	11011 10110 10110	Initialization. LS $A = A - M.$
4	0001 00011 11110	10110 01100 01100	Restore, $Q_0 = 0$ LS $A = A - M$
3	00011 00110 00001	01100 11000 11000	Restore, $Q_0 = 0$ LS $A = A - M, Q_0 = 1.$
2	00011 11110	10010 10010	LS $A = A - M$
1	00011	10010	Restore, $Q_0 = 0$
1	0011	00100	LS
1	00010	00101	$A = A - M.$

$\therefore Q = 00101 = 5.$

$A = 00010 = 2.$

Comparison: Restoring division needs an extra "restore" addition every time the trial subtraction goes negative, effectively doubling operations in the worst case. Non-restoring division never undoes a step mid-way — it simply alternates between add and subtract based on the sign of the previous remainder, needing at most one corrective addition at the very end. This roughly halves the average number of arithmetic operations, so non-restoring division is preferred for high-speed processors, despite slightly more complex control logic.

Question 5 — IEEE 754: -42.375 and Addition with $+15.125$

$$5) \quad 42 = 101010_2$$

$$0.375 = 0.011_2$$

$$-42.375 = -101010.011$$

Normalize;

$$1.01010011 \times 2^5$$

$$\rightarrow \text{Exponent} = 5 + 127 = 132$$

$$\Rightarrow 10000100_2$$

$$\rightarrow \text{Mantissa} = 01010011000000000000$$

$$+15.125$$

$$\Rightarrow 15 = 1111_2$$

$$0.125 = 0.001_2$$

Normalize;

$$1.111001 \times 2^3$$

$$\rightarrow \text{Exponent} = 3 + 127 = 130$$

$$\Rightarrow 10000010_2$$

$$\therefore \text{Final result} = -27.25$$

Interpretation: No rounding was needed here since the exact difference fit within the available bits, but in general, shifting the smaller operand's mantissa during alignment can push bits off the end of the register, requiring rounding — the main source of precision loss in IEEE 754 addition. Double precision reduces this loss compared to single precision because it retains 29 more mantissa bits.